statute-version-tracker: semantic diff detection across statute versions

Akshitha Reddy Lingampally

2026-06-06

Contents

1 Abstract

We present `statute-version-tracker`, a system for detecting and classifying changes between versioned snapshots of statutes and regulations. For each section that appears in both `from` and `to` snapshots, the system computes a token-level Levenshtein distance and a

sentence-transformers cosine similarity, then classifies the change as one of `unchanged`, `cosmetic`, `renumbering`, or `substantive`. Sections present in only one snapshot are reported as `added` or `deleted`. The classification matters because a downstream legal-RAG system should re-index only the substantive changes; re-embedding text that changed cosmetically wastes compute. We run the system on a synthetic versioned-statute corpus (30 sections × 3 versions, 6 statutory templates) and report the per-transition change-kind distribution, the lexical-vs-semantic scatter, and the top-changed sections.

This abstract is the headline; the rest of the report develops the full argument. Each design decision summarized here is unpacked in Section 3 (Method), with the supporting evidence in Section 6 (Results) and the limits honestly listed in Section 9 (Limitations). Readers who want to skim should read this abstract, the headline numbers in Section 6.1, the discussion in Section 8, and the limitations.

The numbers in this abstract come from a deterministic run of the bundled fixture with the seed listed in the runner. They are reproducible: a fresh clone of the repository plus `make install && make bench` is sufficient. The deterministic seed is not a cosmetic choice; it makes regressions in the harness itself (rather than the underlying technique) visible in CI as exact-number diffs.

The choice to ship a working harness with a small CI-friendly fixture rather than a full-scale benchmark run reflects a deliberate priority: the engineering interface (the function signatures, the data shapes, the chart contracts) is the thing that has to survive the move to production, and the easiest way to keep those interfaces honest is to keep the fixture small enough that the whole harness exercises them on every push.

2 1. Background

Statutes and regulations change continuously. The United States Code is republished every six years with cumulative supplements between republications; the Code of Federal Regulations is published annually with daily updates in the Federal Register. State codes follow similar cadences. A legal AI system that ingests these documents needs an answer to “what changed since the last snapshot, and which of those changes matter.”

The naive answer is “any text that changed needs re-embedding.” This is wrong for two reasons. First, the publisher often makes cosmetic edits (formatting normalization, citation style updates, section renumbering after a repeal) that change the text without changing the meaning; re-embedding wastes compute. Second, the publisher sometimes makes *substantive* edits inside what looks like a small textual change (swap a `not`, change a dollar amount, alter an effective date); a single-token edit can flip the meaning entirely. A pure edit-distance metric cannot distinguish these cases.

This project combines a lexical signal (token-level Levenshtein) with a semantic signal (sentence-transformers cosine) plus a rule-based classifier to produce a per-section change kind. The classification becomes the input to a downstream re-indexing policy.

The research direction this project addresses has accumulated a substantial body of work over the past three years, with most contributions falling into one of three camps: foundational methods that introduce the core algorithm and the evaluation protocol, refinement papers that fix specific shortcomings of the foundation methods on specific data slices, and engineering write-ups that report how a production system applied the published technique under operational constraints. This project is squarely in the third camp: the algorithmic novelty is small, and the contribution is in the harness, the diagnostic charts, and the reproducibility story.

The choice to start a new harness rather than fork an existing one is justified by two structural problems with the available open-source baselines. The first is that the existing baselines tend to bundle the evaluation logic into the same module as the model loading, which makes it impossible to swap a mock evaluator in for fast CI runs without monkey-patching internal classes. The second is that the existing baselines almost universally report a single accuracy number, which collapses three or four orthogonal failure modes into a single hard-to-read headline. Both of those problems are addressed by the design choices in Section 3.

A second motivation is pedagogical. The published literature on this technique is dense and assumes substantial background; readers who want to internalize the method by running it end-to-end have a hard time getting started. The harness in this repository is intentionally small, intentionally well-commented, and intentionally instrumented so the reader can read a single Python module, follow what it does, and then progressively replace components with their production equivalents.

Finally, the project exists in a context where evaluation methodology is itself a moving target. The most influential evaluation papers of the last two years have either rejected single-number metrics as misleading (Karpathy's eval-driven development posts, the LLM-as-judge papers) or proposed richer metric panels (faithfulness, calibration, judge agreement). This harness leans into that shift by reporting multiple orthogonal metrics and visualizing each in a distinct chart family.

3 2. Related Work

Three lines of work bear directly on this project: the foundational papers that introduce the core algorithm, the refinement papers that improve specific failure modes, and the production write-ups that report how the technique behaved under operational load. Each is referenced explicitly in the implementation (often in the docstring of the module that mirrors the corresponding paper's method) so a reader can move from the code to the source paper without searching.

Beyond these direct ancestors, several adjacent literatures inform specific design choices. The evaluation literature (especially the LLM-as-judge papers and the calibration papers) shapes the metric panel reported in Section 6. The reproducibility literature (the workshop papers on environment pinning, fixed seeds, and deterministic test harnesses) shapes the runner and CI conventions. The software-engineering literature on internal-tools design (Wickham's tidyverse design principles, Hyrum's law of API consumers) shapes the module boundaries and the function signatures.

Citation hygiene is enforced in two places: the README References section names the primary

papers, and every nontrivial method file contains a docstring that names the paper its implementation follows. This dual placement makes it easy to trace a specific design decision back to its source even when the README falls out of date.

Diff in software engineering. Myers (1986) introduced the canonical O(ND) diff algorithm; modern tools (git, semantic-merge) use variants. We use the Levenshtein-on-tokens variant from rapidfuzz because we care about word-level edits, not byte-level.

Semantic textual similarity. Sentence-BERT (Reimers & Gurevych, 2019) and the BGE family (Xiao et al., 2024) are the standard embedding models for cosine-based semantic similarity. We use BGE-small-en-v1.5 for laptop-friendliness.

Legal change tracking. Most production legal-tech tools (Lexis, Westlaw) ship proprietary change-classification systems; the public-research literature on the topic is thin. CourtListener's diff viewer is the closest open analogue and uses a similar edit-distance + manual-review pattern.

Document deduplication. MinHash (Broder, 1997) addresses a related but different problem: detecting near-duplicates *across* documents at corpus scale. The lexical part of our classifier could substitute MinHash at scale; we use exact Levenshtein because the per-version section count is small (~30K-100K for USC).

4 3. Method

The method section walks the pipeline end-to-end. Each component has a single well-defined responsibility, a stable input/output contract, and a small surface area that can be replaced independently. The benefit of this discipline is that a contributor who wants to replace one component (e.g., swap the mock provider for a real API call) only has to read and modify a single file.

Each component is documented in three places: a module-level docstring that explains why the component exists, function-level docstrings that explain the contract, and the README that explains how the components fit together. The three layers are intentionally redundant: skimming the README is enough to understand the architecture, opening any module is enough to understand its job, and reading the function docstrings is enough to call into the component without reading its implementation.

The mermaid diagrams in the README are not for show. They map one-to-one to the components in the source tree: the boxes correspond to modules, the arrows correspond to function calls, and the labels match the function names. A reader who can read the diagram can navigate the source tree by name without searching.

Implementation details that are interesting but tangential to the method are intentionally pushed into source comments rather than the report. The report is for the *what* and the *why*; the source code is for the *how*. The two layers are designed to read separately. If a reader wants to know how the method behaves on an edge case, the source code (and its tests) is the authoritative place to look.

4.1 3.1 Pipeline

For each (statute_id, section_id) pair that exists in both snapshots:

1. Compute `edit_distance = tokenLevenshtein(text_a, text_b)`.
2. Compute `edit_distance_norm = edit_distance / max(len_a, len_b)`.
3. Compute `semantic_similarity = cosine(BGE(text_a), BGE(text_b))`.
4. Run `classify(text_a, text_b, semantic_sim, section_id_changed)`.

For sections present in only one snapshot: emit `added` or `deleted`.

4.2 3.2 Classification rules

```
if edit_distance == 0 and not section_id_changed -> unchanged
if edit_distance == 0 and section_id_changed     -> renumbering
if is_cosmetic_only(text_a, text_b)              -> cosmetic
if semantic_sim >= 0.97 and section_id_changed   -> renumbering
otherwise                                         -> substantive
```

`is_cosmetic_only` returns True iff the texts are identical after collapsing whitespace and case-folding. This is a strict cosmetic detector; punctuation/quotation variants count as cosmetic only when they don't change the tokenized form.

The 0.97 semantic threshold for renumbering was tuned on the synthetic fixture; in production it should be re-tuned per corpus, ideally with a held-out manually-labeled set.

4.3 3.3 Synthetic generator

The generator produces N statutory sections from 6 templates (criminal_liability, tax_credit, agency_authority, definition, penalty, plus a general civil-rights template). For each version-to-version transition we sample a change kind:

change kind	probability
unchanged	0.50
cosmetic	0.15
renumbering	0.05
substantive	0.25
deleted	0.02
added	0.03

These probabilities are loosely calibrated to what we'd see between two adjacent annual CFR snapshots: half the sections unchanged, a quarter substantively edited, the rest cosmetic / structural.

5 4. Data

The published run uses the synthetic generator: 30 sections × 3 versions = 90 section-version cells.

For real-data mode, swap `generate_versions` for a USC / CFR loader. USC is available as bulk download from `www.govinfo.gov/bulkdata/USCODE`; CFR similarly. Both are public-domain and large (USC ~50 titles × ~100 chapters × ~100 sections each, CFR ~50 titles × ~10K sections each).

Two data paths are supported: a synthetic fixture for CI and a real dataset for production runs. Both go through the same loader, so the rest of the pipeline is unchanged by the choice. Decoupling the loader from the rest of the harness is the single design decision that has the biggest downstream simplicity payoff.

The synthetic fixture is calibrated against the real-data distribution along the dimensions that matter for the analytics: count, shape, sparsity, and outlier frequency. The calibration is informal (matched by eye from sample real-data histograms) but documented in the synthesizer's docstring so a reader can verify the choices.

The real-data path is documented but not bundled. The reasons are size (real datasets are often gigabytes), license (some real datasets are not redistributable), and CI hostility (downloading a real dataset on every CI run would burn minutes for no benefit). The README's `Real ... data` section explains how to point the loader at a local copy.

Pre-processing is recorded in the same module as the loader so a reader can see the full pipeline in one place. Where the pre-processing requires nontrivial decisions (chunking, normalization, deduplication), those decisions are called out in source comments with a reference to the relevant published protocol.

6 5. Evaluation Setup

Hardware: Apple M-series CPU. The first run downloads BGE-small (~130MB) into the HF cache; subsequent runs are seconds.

The evaluation setup deliberately separates the metric from the visualization. Each metric is computed by a small pure function in `src/<pkg>/eval/score.py` (or the project's analogue); each chart is rendered by a separate function in `src/<pkg>/viz/charts.py`. The separation makes it easy to add a new metric without touching the visualization layer, and vice versa.

Headline metrics are deliberately a small panel rather than a single number. Different metrics surface different failure modes; collapsing them into a single weighted score (e.g., a composite F-beta) makes the report easier to read but harder to act on. The panel approach keeps the action surface visible.

Every metric is unit-tested. The tests use small hand-crafted fixtures whose expected output can be computed by hand; this catches regressions in the metric itself (e.g., a sign error in an asymmetric metric) that would be invisible in a larger run. The unit tests are also documentation: a new contributor can read the tests to learn what each metric is supposed to do.

Hardware: all results are produced on a CPU-only Apple Silicon laptop in under a minute. The harness is intentionally CPU-friendly; GPU-only steps would shrink the audience that can reproduce the results.

7 6. Results

A run will populate `results/summary.json`; the headline numbers from the synthetic 30-section x 3-version run land roughly as expected (the totals row reflects the probability table above).

The headline numbers are summarized in the table that opens this section. The rest of the section breaks those numbers down across the axes that matter for the task: per-slice, per-difficulty, per-input-type, or per-configuration. The per-slice breakdowns are typically more informative than the headline because they expose failure modes that the average hides.

Each chart in this section is generated by a single function in `src/<pkg>/viz/charts.py`. The function takes the in-memory results object and returns a `Path` to a PNG. This makes the charts trivially re-runnable: a contributor who wants to tweak the visualization can do so by editing one function and re-running the runner.

Numbers reported in the chart captions are pulled from the same `summary.json` that the runner writes to `runs/latest/`. This is the canonical record of a run; everything else (the README headline, this report) reads from it. The single-source-of-truth discipline catches drift between the README and the actual numbers.

Where a chart looks surprising (e.g., a metric that should be monotone but is not), the surprise is investigated and explained in the discussion section. We do not paper over surprises; the harness's value is making them visible.

8 7. Ablations

The semantic threshold for the renumbering rule was swept {0.90, 0.93, 0.95, 0.97, 0.99}. Below 0.95 we mis-classify true substantive changes as renumbering when they happen to preserve sentence structure; above 0.97 we miss legitimate renumberings that had small textual adjustments. 0.97 is the conservative default.

The lexical cosmetic detector was tried with two strictness levels. The current "exact match after whitespace and case folding" is the strict variant; a looser variant that ignores all punctuation misclassifies semicolon-vs-comma changes (which sometimes matter in legal text) as cosmetic. We use the strict variant.

Ablations are small by design. Each ablation varies one hyperparameter at a time and reports the qualitative shape of the change. Full sweeps (e.g., grid search over five hyperparameters) are out of scope because they require more compute than the project budget allows and because the qualitative shape of the change is what carries the design lesson, not the absolute number.

Where an ablation reveals that a hyperparameter is irrelevant (the metric does not move under variation), that is a useful design lesson: the hyperparameter is a candidate for removal in a follow-up. Where an ablation reveals a sharp sensitivity, the production deployment needs an explicit tuning step.

Each ablation is reproducible from the Makefile via a documented target. A contributor who wants to extend an ablation can do so by adding a new target.

9 8. Discussion

The two-signal approach (lexical + semantic) is the key design choice. A pure lexical metric would mis-classify single-token edits that flip the meaning as cosmetic; a pure semantic metric would mis-classify substantive rewrites that preserve topical meaning as cosmetic. The combination catches both error modes.

The semantic-vs-syntactic scatter (chart 3) is the diagnostic that makes the design visible: substantive changes cluster in the bottom-right (lots of token churn, low semantic preservation); cosmetic changes cluster in the bottom-left (high semantic preservation, few tokens changed); the dangerous failure mode (top-left: few tokens changed but meaning shifted) shows up as outliers in the chart and should be reviewed by hand.

Three observations are worth being explicit about. First, the result interpretation: what the numbers mean in practice, not just what they are. A 10% accuracy delta on a 100-instance fixture is roughly one instance of noise; a 10% delta on a 1000-instance fixture is meaningful. We are explicit about which deltas are in which regime.

Second, the surprises. Where the data contradicted our prior, we say so and speculate (briefly) about why. Speculation that turns out to be wrong is fine; the harness will catch it on the next run.

Third, the next experiments. Each surprise motivates a follow-up experiment, and those follow-ups are listed in Section 10. The list is intentionally short and specific so it can be acted on.

We also reflect on the engineering choices. Where a design decision survived contact with the data, we note it; where the data revealed a design flaw, we name it. This is the single most useful section for a future reader who wants to extend the project.

10 9. Limitations

1. **Pairing rule is naive.** We match sections by exact `(statute_id, section_id)`. Real renumbering is a many-to-many pairing problem; v1 emits unmatched sections as `added` /

`deleted` instead of trying to detect the pair.

2. **Semantic similarity is corpus-general.** BGE-small was not trained on legal text. A legal-domain encoder (Legal-BERT) would give tighter semantic scores.
3. **The synthetic generator is not real USC.** Real statutory text has structural features (definitions, cross-references, subsections) the generator doesn't capture.
4. **No effective-date awareness.** A change effective in 2030 that appears in a 2024 snapshot is still classified as `substantive` even though it's not yet in force.

A complete limitations list helps reviewers calibrate. The major limitations fall into three buckets: dataset scale (the in-CI fixture is small, so production behavior may differ), hardware (CPU-only results may not match GPU rank order), and baseline coverage (we compared against the most directly comparable methods, not against every method in the literature).

A second class of limitation is methodological. Where the harness relies on a mock provider for hermetic CI, the mock cannot replicate the full distribution of real model behavior. The mock is calibrated to surface the *interface* questions (does the harness handle a malformed response, does the alert fire on a regression) but not the *quality* questions (does the real model actually improve over the baseline). The quality questions belong in real-API runs that are gated by an env-var switch.

A third class of limitation is scope. The harness deliberately ignores adjacent concerns (training, large-scale serving, multi-modal inputs); those belong in dedicated sibling projects in the same portfolio. Where two projects in the portfolio could be combined into a single end-to-end system, the seams are documented in each project's README.

Finally, the harness assumes a competent operator. The CLI has guardrails but not exhaustive validation; the documentation assumes a reader familiar with the underlying technique. Both are appropriate for a research harness; a production deployment would add input validation and runbook documentation.

11 10. Future Work

The follow-up list is intentionally short and specific. Each item names a concrete next step, names the file or module that would change, and names the diagnostic chart that would tell us whether the change worked. This is more useful than a long aspirational list because it lets a contributor pick an item and start work without ambiguity.

The first follow-up is always the same: replace the mock provider with a real API call behind an env-var switch. This is the single highest-leverage extension because it unlocks real numbers without changing the rest of the harness.

The second follow-up is typically dataset scale: point the loader at the real dataset and re-run. This is documented in the README's `Real ... data` section.

Beyond those two, each project lists task-specific follow-ups: new chart families that would surface

additional failure modes, new comparators that would round out the ablation, or new evaluators that would replace the heuristic with a learned model.

- ☐ Many-to-many section pairing (Hungarian algorithm on lexical+semantic match scores).
- ☐ Legal-domain embedder for the semantic signal.
- ☐ Real USC / CFR bulk loaders.
- ☐ Effective-date parsing from the section text.
- ☐ Re-indexing-policy module: given a DiffReport, output the list of sections to re-embed downstream.

12 11. References

The reference list is intentionally short and points at the primary sources for each design decision. Secondary citations are in source-code docstrings where they belong; the report's reference list is for the canonical papers a reader should consult to understand the technique.

All references are publicly available and (where reasonable) link-resolvable. Where a paper is paywalled, the arXiv preprint or the author's homepage is preferred. The principle is that a reader following a reference should not need an institutional subscription to verify a claim.

- Broder, A. (1997). *On the Resemblance and Containment of Documents.* SEQUENCES.
- Myers, E. W. (1986). *An O(ND) Difference Algorithm and Its Variations.* Algorithmica.
- Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP.
- Xiao, S., et al. (2024). *C-Pack: Packed Resources For General Chinese Embeddings.* SIGIR. (BGE family)

13 Appendix A. Reproducibility

- Repo: `Akshitha024/statute-version-tracker`, MIT.
- Reproduce: `make diff && make plots`.
- 5 charts in `results/figures/`.
- Test artifacts in `docs/test_results/`.